RISTORANTINO

Sistema de Gestion de Restaurante

Especificación Técnica del Proyecto

Version 1.0

Fecha: 29 de mayo de 2026

Autores:

Elvis López

Guillermo Dávalos

1. Introduccion

1.1 Descripción General

Ristorantino es un sistema de gestión integral para restaurantes desarrollado con Django. Cubre las operaciones diarias del salón: administración de mesas, toma de pedidos en comanda digital, display de cocina en tiempo real y emisión de documentos tributarios con trazabilidad fiscal completa.

El sistema está diseñado para personal de restaurante (mozos, cajeros, chef) con interfaces diferenciadas por rol, garantizando que cada empleado acceda únicamente a las funciones que le corresponden.

1.2 Objetivos del Sistema

- Digitalizar la operativa del salón: apertura de mesas, toma de pedidos y cierre de cuenta.
- Proporcionar un monitor de cocina con cola FIFO para optimizar tiempos de preparación.
- Emitir documentos tributarios (facturas, recibos) con número de timbrado y trazabilidad de caja.
- Registrar KPIs de margen por producto mediante seguimiento de precio de venta y costo de insumo.
- Mantener un historial de arqueos de caja para conciliación contable.

1.3 Alcance

El sistema abarca los siguientes módulos funcionales:

- Gestión de mesas y sectores del restaurante.
- Ingreso y modificación de pedidos (comanda digital).
- Flujo de cocina con cambio de estados por ítem.
- Facturación fiscal con timbrados y tipos de documento.
- Administracion de caja y arqueos.
- Panel de administración Django para configuración del sistema.

2. Stack Tecnologico

La siguiente tabla resume las tecnologías utilizadas en el proyecto:

Componente	Tecnologia	Detalle
Backend framework	Django 6.0.3	ORM, autenticación, admin, rutas, plantillas
Base de datos	PostgreSQL 15+	127.0.0.1:5432 / db: ristorantino / user: postgres
Frontend CSS	Bootstrap 5.3.0	Grid responsivo, componentes y utilidades
Tema visual	CSS Japandi personalizado	Variables CSS con paleta minimalista Japandi
Servidor de desarrollo	Django runserver	http://localhost:8000
Lenguaje	Python 3.12+	Entorno virtual en env\ (Windows)
Control de versiones	Git	Repositorio local

2.1 Requisitos del Entorno

- Python 3.12 o superior instalado en el sistema.
- PostgreSQL 15+ con una base de datos llamada 'ristorantino' creada previamente.
- Node.js no es requerido (Bootstrap se sirve via CDN).
- Sistema operativo Windows (el entorno virtual usa Scripts\ en lugar de bin\).

3. Arquitectura del Sistema

3.1 Patrón de Diseño

El proyecto sigue el patrón MVT (Model-View-Template) de Django, variante del MVC clásico:

- Model: Define la estructura de datos y lógica de negocio (modelos ORM en pedidos/models.py).
- View: Procesa peticiones HTTP y orquesta la lógica (vistas en pedidos/views.py).
- Template: Genera el HTML final combinando datos con plantillas (pedidos/templates/).

3.2 Estructura de Directorios

La siguiente estructura describe la organización del proyecto:

Ruta	Descripcion
`ristorantino/`	Raiz del proyecto
`  core/`	Configuracion global del proyecto Django
`    settings.py`	Parametros de BD, apps instaladas, templates, static
`    urls.py`	Enrutamiento principal (incluye pedidos.urls)
`    wsgi.py / asgi.py`	Entry points de servidor
`  pedidos/`	Aplicación principal con toda la lógica de negocio
`    models.py`	Definición de todos los modelos ORM
`    views.py`	Vistas HTTP y endpoints AJAX
`    urls.py`	Rutas de la aplicación pedidos
`    admin.py`	Registro de modelos en el panel de administración
`    templates/`	Plantillas HTML de la aplicación
`      base.html`	Layout base (Bootstrap 5.3 + tema Japandi)
`      salon.html`	Grilla de mesas con visualización de sillas
`      comanda.html`	Pantalla de toma de pedidos (dos columnas)
`  static/`	Archivos estáticos del proyecto
`    css/`	Hojas de estilo personalizadas (tema Japandi)
`    js/`	Scripts JavaScript del frontend
`    img/`	Imágenes y recursos gráficos
`  env/`	Entorno virtual Python (no versionado)
`  requirements.txt`	Dependencias Python del proyecto
`  manage.py`	Utilidad de administracion de Django

3.3 Configuración de Base de Datos

Las credenciales de conexión a PostgreSQL están configuradas en core/settings.py:

- Motor: django.db.backends.postgresql
- Host: 127.0.0.1 | Puerto: 5432
- Nombre de base de datos: ristorantino
- Usuario: postgres / Contrasena: admin

También existe un archivo db.sqlite3 en el directorio raíz para uso local sin PostgreSQL, aunque la configuración de settings.py apunta a Postgres por defecto.

4. Modelo de Datos

El modelo de datos está organizado en cinco grupos funcionales que reflejan los distintos dominios del negocio restaurantero.

4.1 Acceso y Personal

Modelo	Campos clave	Descripcion
Rol	nombre (str)	Define el rol del empleado. Valores típicos: mozo, cajero, chef.
Empleado	user (OneToOne -> User), rol (FK -> Rol)	Extiende el User de Django con un rol. Acceso: user.empleado.rol.nombre

4.2 Infraestructura y Menu

Modelo	Campos clave	Descripcion
Sector	nombre (str)	Agrupa mesas por zona del restaurante (Ej: salón principal, terraza, privado).
Mesa	numero, sector (FK), estado (choices)	Unidad física del restaurante. Cuatro estados: Libre, Ocupada, Reservada, Sucia.
Categoria	nombre (str)	Clasificación de productos del menú (Ej: Entradas, Platos principales, Bebidas).
Producto	nombre, categoría (FK), precio_venta, costo_insumo, activo	Item del menú. Almacena precio_venta y costo_insumo para calcular KPIs de margen bruto.

4.3 Pedidos

Modelo	Campos clave	Descripcion
Cliente	nombre, teléfono, email	Información del cliente. Un cliente por defecto se asigna al abrir una mesa sin datos.
Pedido	mesa (FK), cliente (FK), estado, total, fecha_apertura	Pedido activo en una mesa. actualizar_total() usa F-expressions para sumar cantidad * precio_historico.
PedidoDetalle	pedido (FK), producto (FK), cantidad, precio_historico	Linea de detalle. precio_historico es un snapshot del precio al momento del pedido.

4.4 Cocina

Modelo	Campos clave	Descripcion
FlujoEstado	detalle (FK -> PedidoDetalle), estado (choices), timestamp_creacion, timestamp_cambio	Rastrea cada item de pedido a través de los estados de cocina. Permite cola FIFO ordenando por timestamp_creacion ascendente.

4.5 Contabilidad y Documentos Tributarios

Modelo	Campos clave	Descripcion
TipoDocumento	nombre, código (str)	Define los tipos de comprobante fiscal (Factura, Recibo, Nota de crédito, etc.).
Timbrado	número, fecha_inicio, fecha_fin, activo	Número de timbrado habilitado por la autoridad fiscal. Controla la vigencia de emisión.
Caja	nombre, saldo_inicial, activo	Representa una caja registradora física. Puede asociarse a múltiples timbrados.
CajaTimbrado	caja (FK), timbrado (FK), numero_desde, numero_hasta, ultimo_numero	Vincula una caja a un timbrado con el rango de numeración autorizado.
Arqueo	caja (FK), fecha, saldo_apertura, saldo_cierre, responsable (FK)	Registro de conciliación de caja al inicio y cierre del turno.
DocumentoTributario	tipo (FK), caja_timbrado (FK), número, fecha, total, cliente (FK)	Comprobante fiscal emitido. Vincula el pedido al timbrado, número correlativo y cliente.
PedidoDocumento	pedido (FK), documento (FK)	Tabla puente que vincula un Pedido con su DocumentoTributario correspondiente.

5. Estados de Mesa

Cada mesa transita por los siguientes estados durante su ciclo de vida operativo:

Estado	Significado	Comportamiento del sistema
Libre	Mesa disponible para ser asignada	Estado inicial. El botón 'Abrir Mesa' está habilitado.
Ocupada	Mesa con un pedido activo en curso	Generado por abrir_mesa(). Redirige si ya existe pedido activo.
Reservada	Mesa con reserva pero sin pedido activo	Gestionado manualmente o por módulo de reservas futuro.
Sucia	Mesa liberada pendiente de limpieza	Transición post-cierre de pedido. Requiere acción del personal para liberar.

Diagrama de transición de estados:

```
Libre -> [abrir_mesa()] -> Ocupada -> [cerrar pedido] -> Sucia -> [limpiar] -> Libre
```

La transición a Reservada puede originarse desde Libre, gestionada por el módulo de reservas o por acción manual del administrador.

6. Flujo de Cocina

6.1 Estados del FlujoEstado

Cada ítem (PedidoDetalle) genera un registro FlujoEstado que avanza secuencialmente. La vista monitor_cocina() muestra los items ordenados por timestamp_creacion ascendente (FIFO) para que los pedidos mas antiguos se atiendan primero.

Estado origen	Estado destino	Evento	Notas
Pendiente	Cocina	Empleado de cocina acepta el ítem	FlujoEstado creado al agregar item al pedido.
Cocina	Listo	El plato está preparado	Se registra timestamp_cambio. Visible en monitor.
Listo	Entregado	Mozo entrega el plato en mesa	Item sale del monitor de cocina.
Entregado	(final)	Estado terminal del item	Se conserva para auditoría y KPIs.

6.2 Diagrama de Flujo

```
[Item creado]
     |
     v
 PENDIENTE  -->  COCINA  -->  LISTO  -->  ENTREGADO
              (chef)       (chef)      (mozo)
```

6.3 Lógica de cambiar_estado_cocina()

- Recibe el ID del FlujoEstado y determina el siguiente estado en la secuencia.
- Actualiza el campo estado y registra el timestamp_cambio con la hora actual.
- Retorna una respuesta JSON con el nuevo estado para actualizar la UI sin recargar.
- Los ítems en estado 'Entregado' desaparecen del monitor de cocina.

7. Vistas y Endpoints

Las siguientes vistas componen la interfaz funcional del sistema. Todas residen en pedidos/views.py y están enrutadas desde pedidos/urls.py.

Vista / Funcion	URL / Metodo	Descripcion
`salon_principal()`	GET /salon/	Grilla de mesas por sector con modales para acciones rápidas.
`abrir_mesa()`	POST /mesa/abrir/	Abre una mesa, crea Pedido con cliente por defecto; redirige si ya está ocupada.
`tomar_pedido()`	GET/POST /pedido/<id>/	Pantalla de ingreso de pedidos (comanda.html), layout dos columnas menu vs. pedido.
`agregar_producto_ajax()`	POST /pedido/agregar/	Endpoint AJAX; incrementa cantidad si el ítem existe o crea nuevo PedidoDetalle; retornar JSON.
`monitor_cocina()`	GET /cocina/	Display de cocina; items ordenados por más antiguos primero (FIFO).
`cambiar_estado_cocina()`	POST /cocina/estado/	Avanza FlujoEstado para un ítem de pedido.

7.1 Endpoint AJAX: agregar_producto_ajax()

Este endpoint es el núcleo de la comanda digital. Su lógica de negocio es:

- Recibe el ID del pedido y el ID del producto via POST.
- Busca si ya existe un PedidoDetalle para ese producto en el pedido activo.
- Si existe: incrementa el campo 'cantidad' en 1 y llama a actualizar_total().
- Si no existe: crea un nuevo PedidoDetalle y crea el FlujoEstado inicial en 'Pendiente'.
- Retorna un objeto JSON con los datos actualizados para refrescar la UI sin recargar.

8. Frontend y Plantillas

8.1 Layout Base (base.html)

Todas las vistas heredan de base.html, que incluye:

- Bootstrap 5.3.0 via CDN para el sistema de grilla, componentes y utilidades.
- Variables CSS personalizadas del tema Japandi (paleta minimalista con tonos neutros).
- Barra de navegación con acceso rápido según el rol del usuario autenticado.
- Bloque de mensajes Django para notificaciones de acciones (alertas Bootstrap).

8.2 Salon Principal (salon.html)

La vista del salon utiliza CSS Grid para renderizar las mesas con visualización de sillas:

- Cada mesa se representa como una tarjeta con color segun su estado (verde=Libre, rojo=Ocupada, amarillo=Reservada, gris=Sucia).
- Las sillas se dibujan alrededor de la mesa usando posicionamiento CSS absoluto.
- Modales Bootstrap permiten acciones rápidas (abrir mesa, ver pedido) sin navegar a otra página.

8.3 Comanda (comanda.html)

La pantalla de toma de pedidos usa un layout de dos columnas:

- Columna izquierda: catálogo de productos agrupados por categoría, con botón de agregar.
- Columna derecha: resumen del pedido actual con cantidades, precios históricos y total.
- La interacción usa fetch() de JavaScript para llamar a agregar_producto_ajax() y actualizar la columna derecha sin recargar la página.

9. Autenticación y Control de Acceso

9.1 Modelo de Autenticacion

El sistema extiende el modelo de autenticación nativo de Django:

- Cada empleado tiene un User de Django con username y password para el login.
- El modelo Empleado está vinculado al User mediante una relación OneToOne.
- El rol del empleado se accede con la expresión: user.empleado.rol.nombre

9.2 Roles del Sistema

Rol	Descripcion	Acciones permitidas
mozo	Personal de salon	Abrir mesas, tomar pedidos, marcar items como entregados.
cajero	Personal de caja	Emitir documentos tributarios, gestionar caja y arqueos.
chef	Personal de cocina	Ver monitor de cocina, cambiar estados de ítems (Pendiente -> Cocina -> Listo).

Las vistas aplican el decorador @login_required de Django y verifican el rol según la lógica de negocio de cada función.

10. Comandos de Operación

10.1 Configuración Inicial y Desarrollo

Comando	Descripcion
`env\Scripts\activate`	Activa el entorno virtual Python (Windows)
`pip install -r requirements.txt`	Instala todas las dependencias del proyecto
`python manage.py makemigrations`	Genera archivos de migración a partir de los modelos
`python manage.py migrate`	Aplica las migraciones a la base de datos PostgreSQL
`python manage.py showmigrations`	Muestra el estado de todas las migraciones
`python manage.py createsuperuser`	Crea un usuario administrador para el panel Django
`python manage.py runserver`	Inicia el servidor de desarrollo en http://localhost:8000
`python manage.py shell`	Abre la shell interactiva de Django para pruebas

10.2 Panel de Administración

Django Admin esta disponible en http://localhost:8000/admin/ para usuarios con is_staff=True. Desde allí se pueden gestionar todos los modelos: empleados, mesas, productos, categorías, timbrados, documentos tributarios y arqueos.

11. Glosario

Termino	Definicion
Comanda	Documento o pantalla donde el mozo registra los pedidos de una mesa.
FIFO	First In, First Out. Los pedidos más antiguos se atienden primero en la cola de cocina.
Timbrado	Número habilitado por la SET (Paraguay) para emisión de comprobantes fiscales.
Arqueo	Conteo y conciliación del dinero en caja al inicio o cierre del turno.
FlujoEstado	Entidad que registra el estado actual e historial de transiciones de un ítem de pedido.
precio_historico	Snapshot del precio de venta al momento de crear el detalle del pedido. Inmutable.
F-expression	Expresión de Django ORM que referencia valores de columna directamente en la BD.
MVT	Model-View-Template. Patron arquitectonico de Django (variante de MVC).
ORM	Object-Relational Mapper. Capa de abstraccion de Django que mapea clases Python a SQL.
Bootstrap	Framework CSS para diseno responsivo. Version 5.3.0 usada via CDN.
Japandi	Estilo decorativo que fusiona lo japonés y escandinavo: minimalismo, tonos neutros.
AJAX	(Asynchronous JavaScript and XML) es una técnica de desarrollo web que permite que una página se comunique con el servidor en segundo plano sin recargar toda la página.